IOT 2025/2026 - Challenge 1

Simone Amighini (person code: 10832147)

Implementation

The goal of this challenge is to develop a commercial-style IoT motion sensor for a smart home in Wokwi using a ESP32 microcontroller. The sensor must detect the presence of human motion and measure the ambient light level (illuminance), so that a central controller can decide whether the room lights should be turned on or off.

The device has been programmed to implement a cycle made of seven steps.

STEP 1 (booting and initialization). The device boots and configures the pins; two sensors are attached to the ESP32: a luminescence photoresistor sensor (LDR) and a motion sensor (PIR). Both the sensors are supplied using gpio pins so that it is possible to switch them off when they are not needed. The power supply of the sensors is off at this point.

STEP 2 (sensing). The power supply of the sensors is switched on, then the LDR analogical voltage and the PIR digital value are read and stored into volatile memory. Finally, both sensors are switched off.

STEP 3 (elaboration of read data and preparation of the massage to be transmitted). While the PIR returns directly the value we need, the LDR voltage must be converted to lux before transmitting the message to the controller. Then the message can be composed using the required syntax:

- MOTION_DETECTED-LUMINOSITY:<lux_measure> (if motion is detected)
- MOTION_NOT_DETECTED-LUMINOSITY:<lux_measure> (otherwise)

STEP 4 (network initialization). The WiFi is switched on in station mode and configured.

STEP 5 (transmission). The message prepared before is sent to the broadcast address using a **transmission power of 2dBm**: this has been done because the specification says *"Assume that the ESP32 sink node is always reachable from every sensor node."*

STEP 6 (network deinitialization). The WiFi is switched off.

STEP 7 (deep sleep). The device is configured to wake up after a **deep sleep period of (47%50 + 5) / 10 = 5.2 seconds**. Then the deep sleep is triggered.

In order to realize a timing analysis (see ENERGY CONSUMPTION ESTIMATION – STEP B) the function *micros()* has been invoked at the start of each step: the function returns a relative timestamp expressed in microseconds and a print with the returned values has been inserted at the of STEP 7, immediately before the deep sleep start.

Energy consumption estimation

IMPORTANT NOTE: read the file *readme.txt* and have a look at the *generate_energy_report.py* script to better understand how the data has been retrieved and computed.

The energy estimation has been obtained using a python script, composed of multiple steps.

STEP A. The provided power samples (csv files) are parsed, then the average values of power are calculated for each state as follows (“.” stands for the concatenation operator):

- Idle: average[(powers < 300 mW from sensor_read) . (powers between 100 mW and 300 mWfrom deep_sleep)]
- Sensing: average[(powers > 300 mW from sensor_read)]
- WiFi: average[(powers < 610 mW from sender) . (powers between 600 mW and 610 mW from deep_sleep)]
- Transmission 2dBm: average[(powers between 610 mW and 630 mW from sender)]
- Transmission 19.5dBm: average[(powers between > 630 mW from sender)]
- Deep sleep: average[(powers < 150 mW from deep_sleep)]

These are the computed values:

- Idle = 243.856 mW
- Sensing = 350.162 mW
- WiFi = 605.892 mW
- TX2dBm = 617.883 mW
- TX19.5dBm = 663.109 mW
- Deep sleep = 45.777 mW

STEP B. A csv file containing the values of the timestamp printed by the program is parsed and obtained from simulation. Here it is an example of input:

Idle start	*Sensing start*	*Idle2 start*	*WiFi start*	*TX start*	*Idle3 start*	*Deep sleep start*
1457722	1458341	1458634	1460272	1676994	1686755	1686772
3790942	3791541	3791834	3793504	4021991	4030916	4030933
6132838	6133530	6133807	6135595	6331921	6341451	6341468
8442289	8442901	8443195	8444894	8641276	8651242	8651259
10753640	10754253	10754546	10756247	10952629	10962592	10962609
13064742	13065424	13065652	13067412	13263601	13273131	13273148
15374790	15375368	15375679	15377329	15572061	15582027	15582044
17684536	17685138	17685432	17687029	17883013	17892542	17892559
...	...	...	...	...	...	...

Then, the average time elapsed by each state is computed as follows:

- Idle: average[(Sensing_start – Idle1_start) + (WiFi_start – Idle2_start) + (Deep_sleep_start - Idle3_start)]

- Sensing: average[Idle2_start - Sensing_start]
- WiFi: average[(TX_start – WiFi_start)]
- Transmission: average[Idle3_start - TX_start]
- Deep sleep: due to Wokwi simulator limitations, its exact value has been determined a priori (see INTRODUCTION – STEP 7)

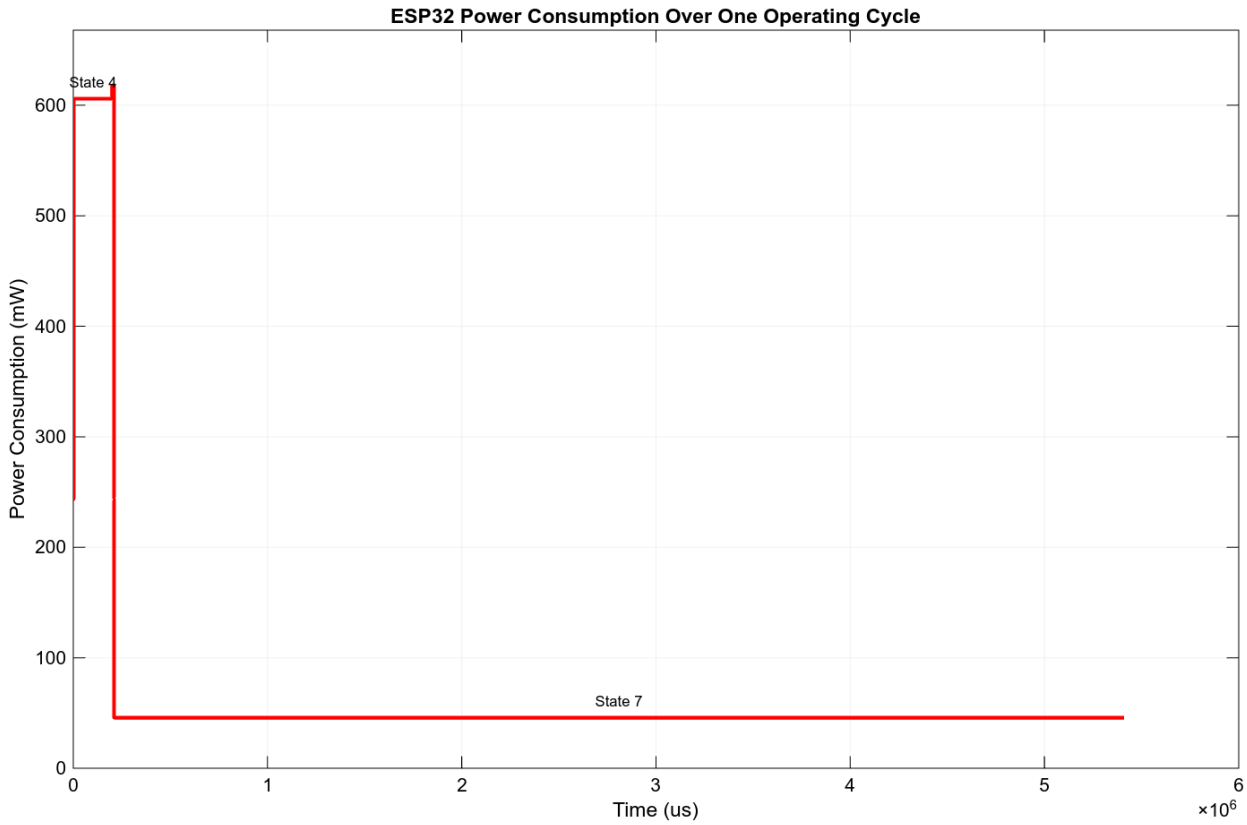
These are the computed values:

- Idle = 2263.059 us
- Sensing = 285.353 us
- WiFi = 197826.882 us
- TX = 9705.235 us
- Deep sleep = 5200000.000 us
- CYCLE PERIOD = 5410080.529 us

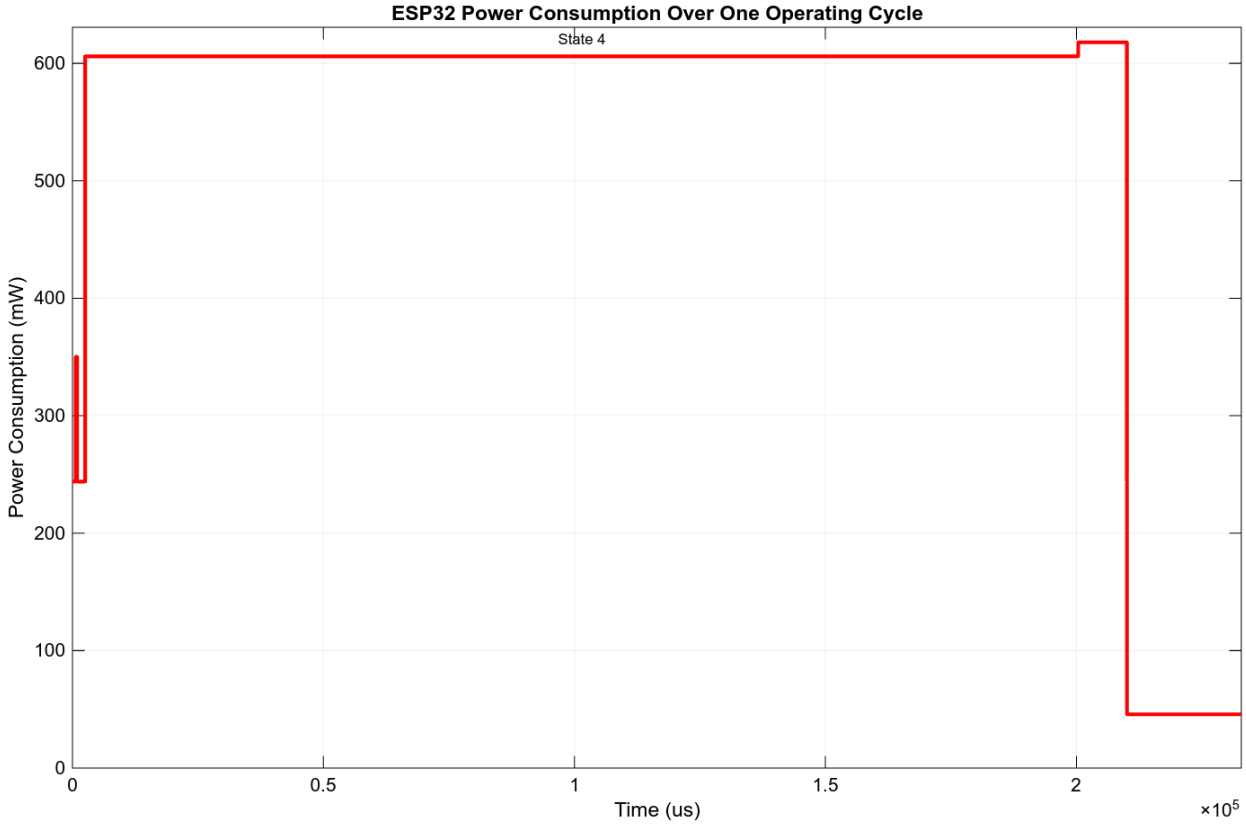
The TX value is referred to the implementation of 2dBm transmission which has been chosen for the system, yet its value can be used also in case of 19.5dBm transmission: anyway, as discussed in the implementation description, there is no need to use 19.5dBm transmission.

Power-time graph (based on computed averages):

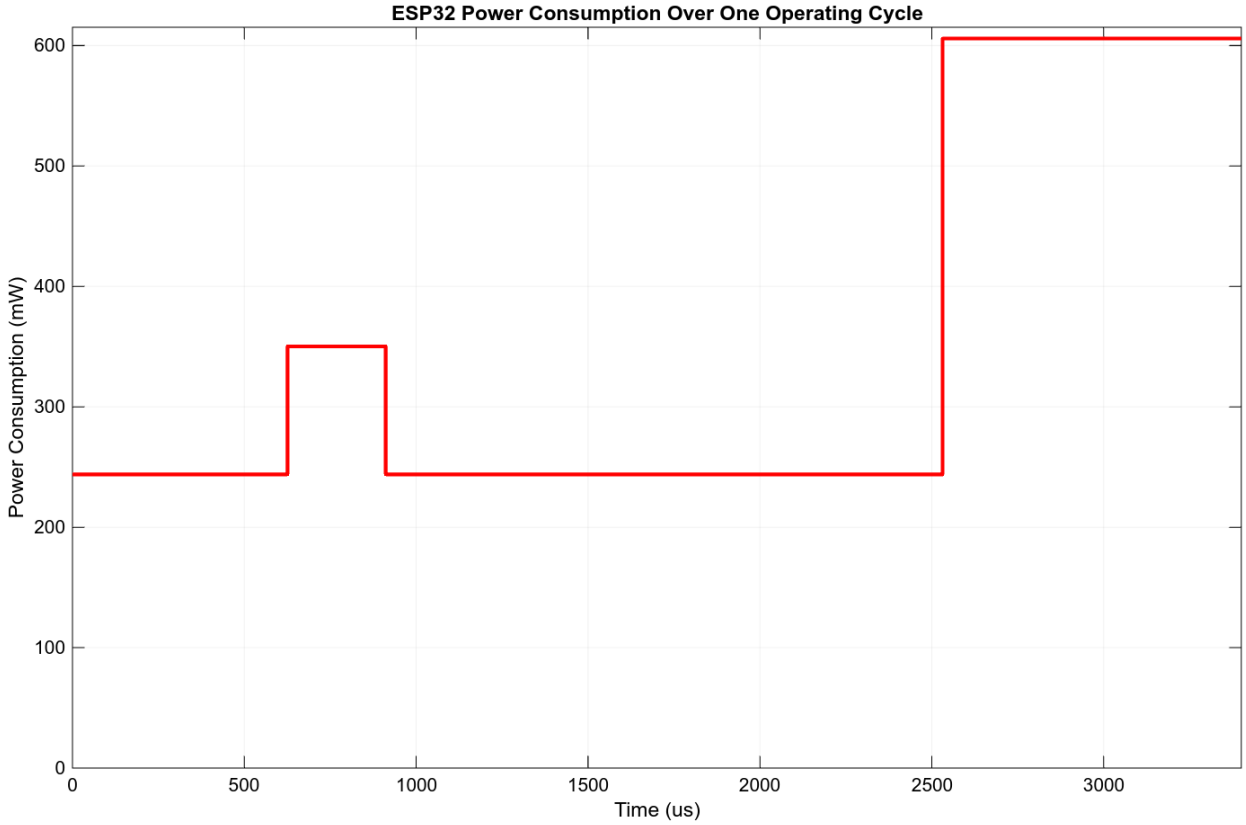
- full graph



- zoom on WiFi and transmission

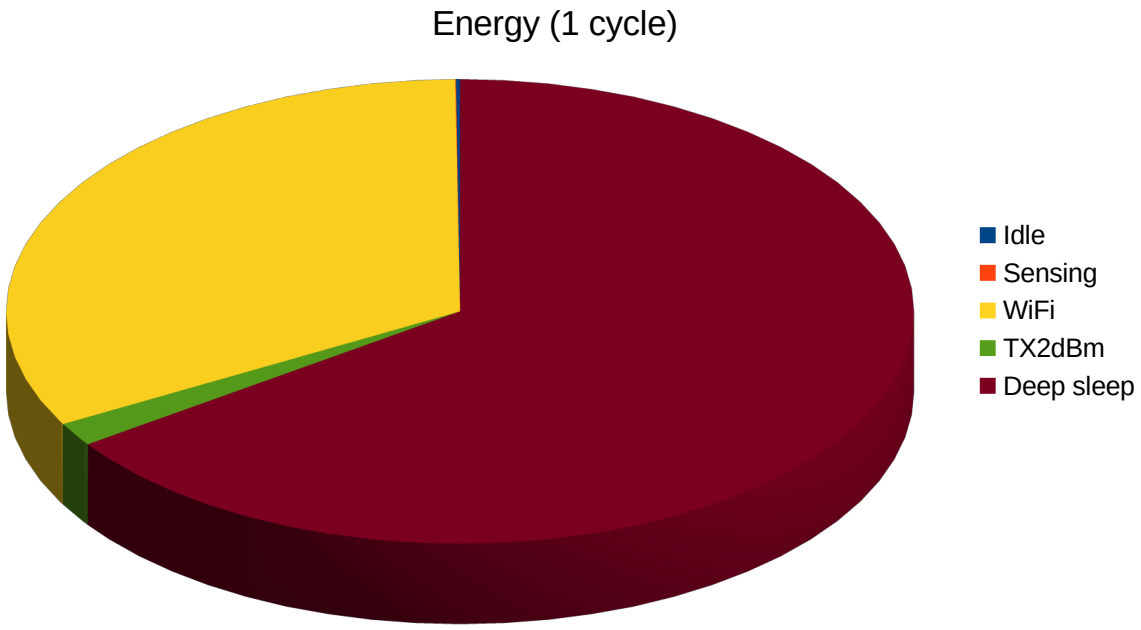


- zoom on boot and sensing



STEP C. The device is equipped with a **battery of capacity (2147%5000 + 15000) = 17147 J**: by multiplying timings and powers, the energy drained by the system is computed. These are the computed values:

- Idle = 0.552 mJ
- Sensing = 0.100 mJ
- WiFi = 119.862 mJ
- TX2dBm = 5.997 mJ
- TX19.5dBm = 6.436 mJ
- Deep sleep = 238.039 mJ
- CYCLE ENERGY (with TX2dBm) = 364.549 mJ
- CYCLE ENERGY (with TX19.5dBm) = 364.988 mJ



Another useful information is the weighted average of the power on one cycle:

- CYCLE POWER (with TX2dBm) = 67.383 mW
- CYCLE POWER (with TX19.5dbM) = 67.464 mW

Finally, the estimated operating lifetime is:

- Cycles per battery (with TX19.5dbM) = 46979
- Battery life (with TX19.5dbM) = 254160.173 s (2.942 days)
- Cycles per battery (with TX2dBm) = 47036
- **Battery life (with TX2dBm) = 254468.548 s (2.945 days)**

Possible improvements

At a first glance, making the deep sleep longer seems to be an easy way to improve the battery life. Here it is a simulation with a period of 10 seconds:

ENERGY CONSUMPTION:	BATTERY LIFE ESTIMATION:
Idle = 0.552 mJ	Battery energy = 17147.000 J
Sensing = 0.100 mJ	Cycles per battery = 29347
WiFi = 119.862 mJ	Battery life = 299635.233 s (3.468 days)
TX2dBm = 5.997 mJ	
Deep sleep = 457.768 mJ	
CYCLE = 584.278 mJ	

Yet this has an important negative consequence: the measures are made with a lower frequence so the system gets less responsive. In my opinion, this is not a good improvement or, at least, more information is needed in order to understand if the tradeoff is convenient.

Another possible improvement is way more effective: the message could be sended only in case it is different from the previous message sent. This allows to save the energy used for WiFi activation and transmission in some cycles: the improvement yet depends on the data read by the sensors. Given that WiFi + TX consumes about 35% of the whole energy in a cycle and calling x the non transmission probability: $E_{CYCLE\_OPTMIZIED}(x) = x \cdot 0.65E_{CYCLE} + (1 - x)E_{CYCLE}$.
For example, if $x = 0.2$ $E_{CYCLE\_OPTMIZIED} = 339{,}031\ mJ$: it is about a –7% improvement on the energy consumption on the entire device lifetime.